ARTIFICAL INTELLIGENT

Name:	Basit Ali
SAP:	54596
Section:	CS 6-1
Submitted to:	Mr Junaid Khan
Date :	12/03/2026
Lab:	07

Riphah International University, I-14 Islamabad

CLI

54596_BaasitAli_lab07.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Insert code cell below (Ctrl+M B)

[7] 32s

```python
def evaluate(b):
    for row in range(0, 3):
        if b[row][0] == b[row][1] and b[row][1] == b[row][2]:
            if b[row][0] == 'x':
                return 10
            elif b[row][0] == 'o':
                return -10

    for col in range(0, 3):
        if b[0][col] == b[1][col] and b[1][col] == b[2][col]:
            if b[0][col] == 'x':
                return 10
            elif b[0][col] == 'o':
                return -10

    if b[0][0] == b[1][1] and b[1][1] == b[2][2]:
        if b[0][0] == 'x':
            return 10
        elif b[0][0] == 'o':
            return -10

    if b[0][2] == b[1][1] and b[1][1] == b[2][0]:
        if b[0][2] == 'x':
            return 10
        elif b[0][2] == 'o':
            return -10

    return 0

if __name__ == "__main__":
    board = [['_' for _ in range(3)] for _ in range(3)]

    print("Enter values for the board (x, o, or _):")
    for i in range(3):
        for j in range(3):
            board[i][j] = input(f"Enter value for position [{i}][{j}]: ").lower()

    print("\nFinal Board State:")
    for row in board:
        print(row)

    value = evaluate(board)
    print("\nThe value of this board is", value)
```

```
Enter values for the board (x, o, or _):
Enter value for position [0][0]: x
Enter value for position [0][1]: o
Enter value for position [0][2]: x
Enter value for position [1][0]: x
Enter value for position [1][1]: o
Enter value for position [1][2]: x
Enter value for position [2][0]: x
Enter value for position [2][1]: -
Enter value for position [2][2]: o

Final Board State:
['x', 'o', 'x']
['x', 'o', 'x']
['x', '-', 'o']

The value of this board is 10
```

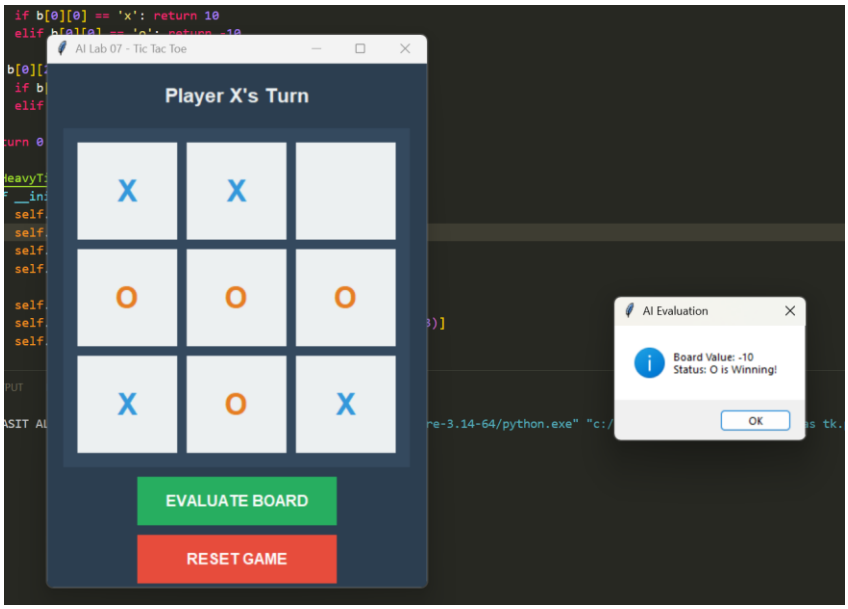
GUI

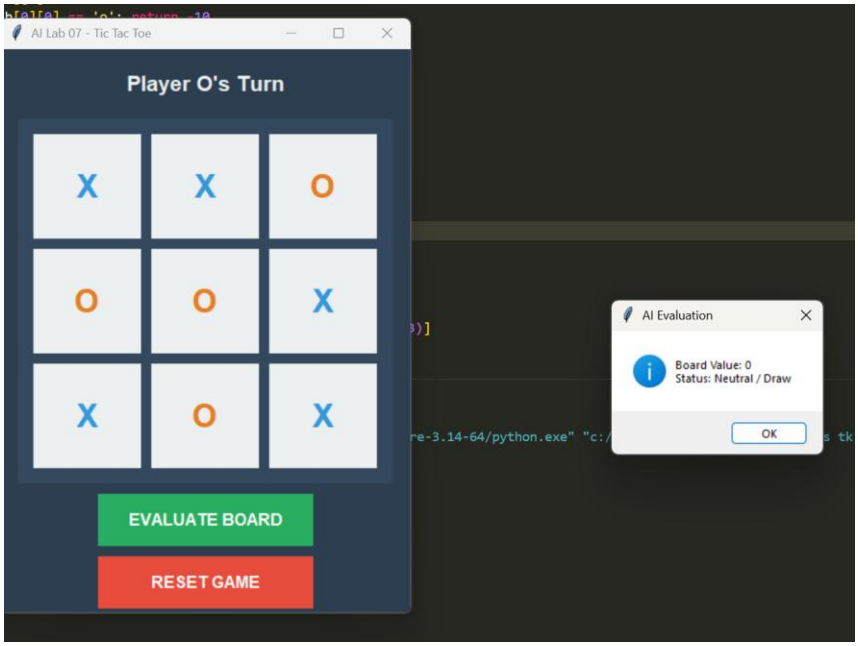
Code

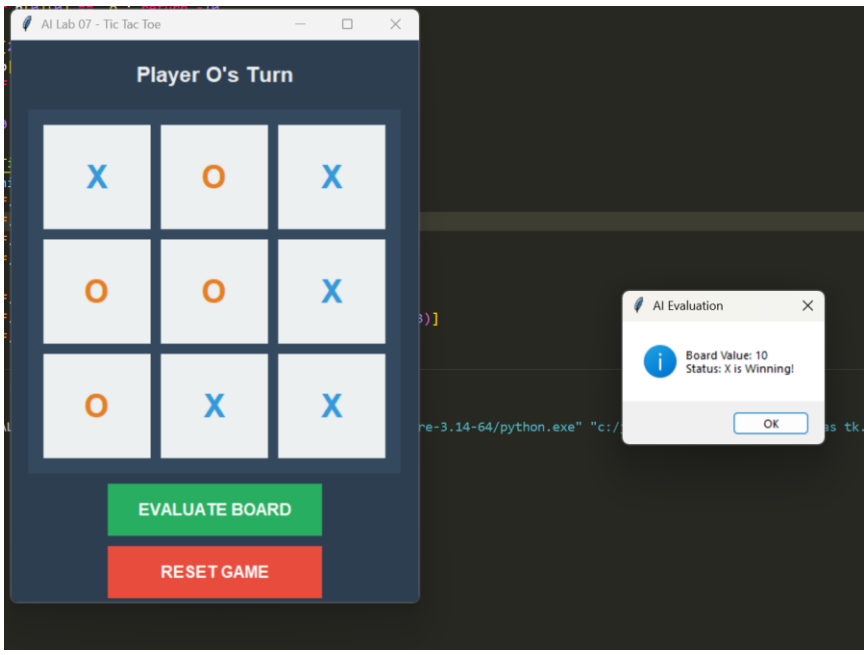
```
import tkinter as tk
from tkinter import messagebox

def evaluate(b):
    for row in range(0, 3):
        if b[row][0] == b[row][1] and b[row][1] == b[row][2]:
            if b[row][0] == 'x': return 10
            elif b[row][0] == 'o': return -10

    for col in range(0, 3):
        if b[0][col] == b[1][col] and b[1][col] == b[2][col]:
            if b[0][col] == 'x': return 10
            elif b[0][col] == 'o': return -10

    if b[0][0] == b[1][1] and b[1][1] == b[2][2]:
        if b[0][0] == 'x': return 10
        elif b[0][0] == 'o': return -10

    if b[0][2] == b[1][1] and b[1][1] == b[2][0]:
        if b[0][2] == 'x': return 10
        elif b[0][2] == 'o': return -10

    return 0

class HeavyTicTacToe:
    def __init__(self, root):
        self.root = root
        self.root.title("AI Lab 07 - Tic Tac Toe")
        self.root.geometry("400x550")
        self.root.configure(bg='#2c3e50')

        self.board = [['_' for _ in range(3)] for _ in range(3)]
        self.buttons = [[None for _ in range(3)] for _ in range(3)]
        self.current_player = 'x'

        self.create_widgets()

    def create_widgets(self):
        self.label = tk.Label(self.root, text="Player X's Turn", font=('Helvetica', 16, 'bold'),
                              bg='#2c3e50', fg='#ecf0f1', pady=20)
        self.label.pack()

        self.grid_frame = tk.Frame(self.root, bg='#34495e', padx=10, pady=10)
        self.grid_frame.pack()

        for r in range(3):
            for c in range(3):
                self.buttons[r][c] = tk.Button(self.grid_frame, text="", font=('Helvetica', 24, 'bold'),
                                               height=2, width=5, bg='#ecf0f1', relief='flat',
                                               command=lambda row=r, col=c: self.make_move(row, col))
                self.buttons[r][c].grid(row=r, column=c, padx=5, pady=5)

        self.score_btn = tk.Button(self.root, text="EVALUATE BOARD", font=('Helvetica', 12, 'bold'),
                                   bg='#27ae60', fg='white', width=20, pady=10, relief='flat',
                                   command=self.show_score)
        self.score_btn.pack(pady=10)
```

```
        self.reset_btn = tk.Button(self.root, text="RESET GAME", font=('Helvetica', 12, 'bold'),
                                   bg='#e74c3c', fg='white', width=20, pady=10, relief='flat',
                                   command=self.reset_game)
        self.reset_btn.pack()

    def make_move(self, r, c):
        if self.board[r][c] == '_':
            self.board[r][c] = self.current_player
            color = '#3498db' if self.current_player == 'x' else '#e67e22'
            self.buttons[r][c].config(text=self.current_player.upper(), fg=color)

            self.current_player = 'o' if self.current_player == 'x' else 'x'
            self.label.config(text=f"Player {self.current_player.upper()}'s Turn")

    def show_score(self):
        score = evaluate(self.board)
        msg = f"Board Value: {score}\n"
        if score == 10: msg += "Status: X is Winning!"
        elif score == -10: msg += "Status: O is Winning!"
        else: msg += "Status: Neutral / Draw"

        messagebox.showinfo("AI Evaluation", msg)

    def reset_game(self):
        self.board = [['_' for _ in range(3)] for _ in range(3)]
        for r in range(3):
            for c in range(3):
                self.buttons[r][c].config(text="", fg='black')
        self.current_player = 'x'
        self.label.config(text="Player X's Turn")

if __name__ == "__main__":
    root = tk.Tk()
    app = HeavyTicTacToe(root)
    root.mainloop()
```

Output




AI Lab 07 - Tic Tac Toe
Player O's Turn
X X O
O O X
X O X
EVALUATE BOARD
RESET GAME
AI Evaluation
Board Value: 0
Status: Neutral / Draw
OK


AI Lab 07 - Tic Tac Toe
Player O's Turn
X O X
O O X
O X X
EVALUATE BOARD
RESET GAME
AI Evaluation
Board Value: 10
Status: X is Winning!
OK